Hacettepe University - Computer Engineering
CMP720 Embedded System Design - Spring 2026

Final Project Topic Proposal Template

(Fill in the sections below. Replace placeholders with your content.)

Group Members

Instructions: List every group member as "Name Surname — Student ID".

- **Umut Kalman — N24120201**
- **Eren Karadağ — N24120203**

Project Topic Title

Instructions: Provide a concise, descriptive title.

Context-Aware Deadline Scheduling Under Memory Contention for Embedded Real-Time Systems

Project Track (choose one)

❑ **Implementation Track**

☑ **Optimization Track**

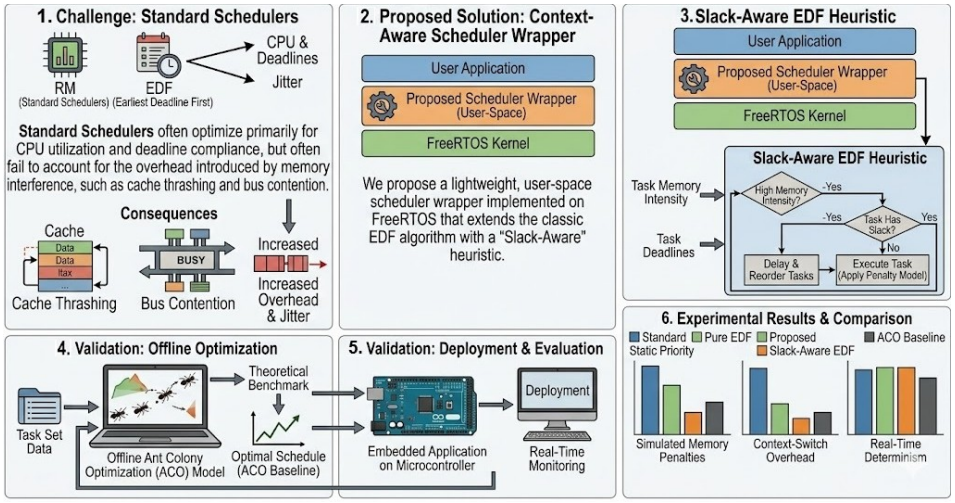
Abstract (300–500 words)

Instructions: In 300–500 words, explain: (i) the problem tackled, (ii) the proposed solution approach and methodology, (iii) high-level system architecture and major components, and (iv) an initial plan and milestones for attacking the problem. Include one crude block diagram (schematic for Implementation track / optimization pipeline diagram for Optimization track) to illustrate the idea.

Modern embedded systems are increasingly required to handle complex, mixed-criticality workloads where computation-bound control loops are run together with data-intensive processing tasks. In these constrained environments, memory contention has come out as a critical factor threatening real-time determinism. Although standard scheduling algorithms, such as Rate Monotonic (RM) and Earliest Deadline First (EDF), optimize primarily for CPU utilization and deadline compliance, they often fail to account for the overhead introduced by memory interference, such as cache thrashing and bus contention. This project presents a context-aware scheduling strategy designed to reduce these penalties in real-time embedded systems.

We propose a lightweight, user-space scheduler wrapper implemented on FreeRTOS that extends the classic EDF algorithm with a "Slack-Aware" heuristic. This mechanism dynamically evaluates the memory intensity of pending tasks and introduces a penalty cost model with metrics to predict contention. With task slack times in mind, the scheduler reorders execution sequences to minimize consecutive memory-heavy operations without violating hard deadlines. To validate the effectiveness of this online heuristic, first we develop an offline Ant Colony Optimization (ACO) model that determines the theoretical optimal schedule for a given task set. Then, we deploy our model in an application running on a microcontroller to examine the custom scheduler we developed.

Experimental results compare the proposed heuristic against standard static priority and pure EDF scheduling, using the ACO solution as a performance baseline. The study demonstrates that the proposed context-aware scheduler significantly reduces simulated memory penalties and context-switch overheads compared to traditional EDF, while maintaining the determinism required for embedded real-time applications.



Motivation / Societal Impact (200–500 words)

Instructions: In 200–500 words, explain why this problem matters for the greater good (societal impact, safety, sustainability, accessibility, cost reduction, reliability, etc.). Include who benefits and how you would measure impact.

Real-time systems play a key role in many safety-critical and infrastructure-level domains such as automotive control units, industrial automation, robotics, avionics, and medical devices. In these environments, timing behavior is directly related to system safety and reliability. Missing a deadline in a braking controller, a robotic arm, or a flight control unit can lead to unstable operation, physical damage, or even life-threatening situations. Although classical scheduling theory provides strong guarantees under ideal assumptions, real-world embedded platforms are far more complex. As modern embedded processors become more multicore and memory-intensive, shared memory contention, cache interference, and bus-level conflicts increasingly affect execution predictability. These effects are often not captured by traditional schedulers such as fixed-priority or Earliest Deadline First (EDF), which mainly focus on CPU utilization and timing constraints.

This project is motivated by a well-known gap between theoretical schedulability and behavior observed in real deployments. In practice, engineers frequently encounter systems that are considered schedulable on paper but still exhibit jitter, sporadic deadline misses, or unstable latency due to hidden resource interference. By introducing scheduling decisions that consider memory interaction patterns, we aim to bring runtime behavior closer to real-world conditions. The focus is on practical and implementable improvements that can be applied on constrained embedded platforms without requiring major system redesigns.

The societal impact of this work is primarily in improving reliability and safety of embedded real-time systems without requiring expensive hardware upgrades. If schedulers can better calculate memory interaction effects, manufacturers may achieve higher reliability margins using the same hardware, reducing cost and energy consumption. This is especially relevant for large-scale deployments like automotive ECUs, smart factories, and IoT infrastructure, where small efficiency gains scale significantly.

Improved timing predictability also supports certification and validation efforts in regulated industries. Standards such as ISO 26262 (automotive) and DO-178C (avionics) require deterministic and explainable behavior under worst-case conditions. By combining deterministic online scheduling logic with an offline optimization reference model (e.g., Ant Colony Optimization used as an analytical baseline), this project also contributes to better engineering methodologies and validation workflows.

Impact will be measured through improvements in deadline miss ratio, execution stability under contention-heavy workloads, and reduced performance variability compared to baseline schedulers. Ultimately, the goal is to move one step closer to real-time systems that behave in the field as predictably as they do in theory, improving safety, trust, and efficiency across embedded computing domains.

Related Work / References

Real-time scheduling has been widely studied, with Earliest Deadline First (EDF) being one of the most well-known dynamic scheduling algorithms. EDF is theoretically optimal for single-processor real-time systems under ideal assumptions and is commonly used in both research and production kernels. However, these guarantees rely on simplified execution models and typically ignore microarchitectural effects such as memory hierarchy interference and shared resource contention.

Recent work shows that shared memory interference can significantly affect timing predictability in multicore real-time systems. Contention on shared caches and DRAM can introduce large execution slowdowns and invalidate traditional worst-case execution time (WCET) assumptions [1, 2]. This highlights the need for interference-aware scheduling models that consider shared resource behavior.

To address this issue, several studies propose memory-centric or contention-aware scheduling strategies. Memory-centric scheduling reduces unpredictability by explicitly modeling memory phases or controlling access patterns at the scheduler level [3]. Similarly, cache-aware and interference-aware approaches explore task placement and co-scheduling strategies that minimize shared resource conflicts in multicore environments [4].

Adaptive and heuristic scheduling techniques have also been explored to extend EDF under non-ideal conditions. For example, hybrid schedulers that dynamically switch between EDF and metaheuristic strategies such as Ant Colony Optimization (ACO) have been proposed to improve performance under overload scenarios [5]. These approaches demonstrate the potential of combining classical scheduling theory with bio-inspired optimization methods. However, many existing methods rely on simplified execution models, assume single-core environments, or introduce non-trivial computational overhead, limiting their applicability in resource-constrained embedded RTOS environments.

References

[1] J. Boudjadar and S. Nadjm-Tehrani, "Schedulability and memory interference analysis of multicore preemptive real-time systems," in *ACM/SPEC International Conference on Performance Engineering*, 2017.

[2] J. M. Aceituno *et al.*, "Interference-aware schedulability analysis and task allocation for multicore hard real-time systems," *Electronics*, 2022.

[3] J. M. Rivas *et al.*, "Implementation of memory centric scheduling for cots multicore systems," in *ECRTS*, 2019.

[4] T. Lugo *et al.*, "A survey of techniques for reducing interference in real-time systems," *IEEE Access*, 2022.

[5] K. Kotecha and A. Shah, "Adaptive scheduling algorithm for real-time operating system," in *IEEE Congress on Evolutionary Computation*, 2008.

How to Submit

- Open the Google Drive sheet (submission tracker): `https://docs.google.com/spreadsheets/d/17ACM2EVBZ9tZY3XIKveWwRSbt-QfVjS7qkYcTWC3GwU/edit?usp=sharing`
- Fill in your group and project details in an appropriate row (make sure you do not delete/edit other students' entries!).
- Create a repository that you will use to submit **all** project work throughout the semester (GitHub or GitLab).
- Upload/commit your proposal files to the repository (both the `.zip` source files with `.tex` and the compiled `.pdf`).
- Paste the repository link into the corresponding column in the sheet.
- If your repository is private, add the instructor as a collaborator: `selmadilek@gmail.com`.